

Modül 3: Tablo Birleştirme (Joins & Unions)

Veri biliminde "Feature Engineering" yaparken farklı veri kaynaklarını bir araya getirmek için Join işlemleri hayati önem taşır.

3.1. Anahtarlar: Birleştirmenin Temeli (Keys)

Tabloları birleştirmeden önce, tablolar arasındaki bağı kuran sütunları anlamalıyız:

- **Primary Key (Birincil Anahtar):** Bir tablodaki her satırı benzersiz şekilde tanımlayan sütun (Örn: müşteri_id).
- **Foreign Key (Yabancı Anahtar):** Başka bir tablonun Primary Key'ine referans veren sütun.

3.2. JOIN Türleri: Yatay Birleştirme

Join işlemleri, iki tabloyu **yan yana** (sütun bazlı) birleştirir.

A. INNER JOIN (Kesişim)

Her iki tabloda da karşılığı olan kayıtları getirir. Eğer bir müşteri hiç sipariş vermediyse, Inner Join sonucunda listelenmez.

```
SELECT musteriler.ad, siparisler.tutar
FROM musteriler
INNER JOIN siparisler ON musteriler.id = siparisler.musteri_id;
```

B. LEFT JOIN (Sol Dış Birleştirme)

Veri biliminde en çok kullanılan Join türüdür. Soldaki tablonun (ilk tablo) tüm kayıtlarını getirir; sağdaki tabloda karşılığı yoksa NULL değerini atar.

- *Neden Önemli?* Kayıp verileri (analiz dışı kalan müşterileri vb.) görmek için kritiktir.

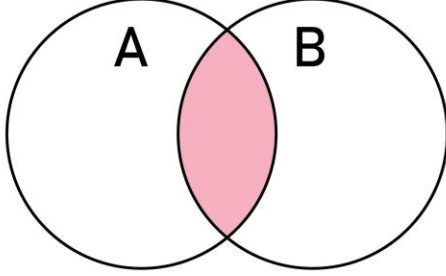
C. RIGHT JOIN

Sağdaki tablonun tüm kayıtlarını getirir. Genellikle Left Join'in tersidir ve çoğu veri bilimci okunabilirlik için tabloların yerini değiştirip Left Join kullanmayı tercih eder.

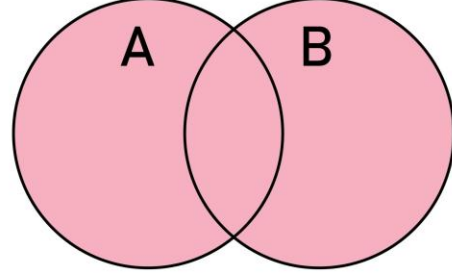
D. FULL OUTER JOIN

Her iki tablodaki tüm kayıtları birleştirir. Eşleşme olmayan yerlere NULL koyar. Veri setindeki tüm "boşlukları" görmek için kullanılır.

Intersection



Union



3.3. Özel Join Durumları

A. Self Join (Kendine Birleştirme)

Bir tablonun kendisiyle birleştirilmesidir. Genellikle hiyerarşik verilerde (Örn: çalışan-yönetici ilişkisi) kullanılır.

```
SELECT a.calisan_adi, b.calisan_adi AS yonetici_adi
FROM calisanlar a
JOIN calisanlar b ON a.yonetici_id = b.id;
```

Modül 3, SQL'in "ilişkisel" (relational) gücünün ortaya çıktığı yerdir. Veri biliminde veriler nadiren tek bir tabloda bulunur; genellikle kullanıcı bilgileri, satışlar ve ürünler farklı tablolardadır. Bu modülde, bu parçaları nasıl birleştireceğimizi derinlemesine işleyeceğiz.

Modül 3: Tablo Birleştirme (Joins & Unions)

Veri biliminde "Feature Engineering" yaparken farklı veri kaynaklarını bir araya getirmek için Join işlemleri hayati önem taşır.

3.1. Anahtarlar: Birleştirmenin Temeli (Keys)

Tabloları birleştirmeden önce, tablolar arasındaki bağı kuran sütunları anlamalıyız:

- **Primary Key (Birincil Anahtar):** Bir tablodaki her satırı benzersiz şekilde tanımlayan sütun (Örn: müşteri_id).

- **Foreign Key (Yabancı Anahtar):** Başka bir tablonun Primary Key'ine referans veren sütun.
-

3.2. JOIN Türleri: Yatay Birleştirme

Join işlemleri, iki tabloyu **yan yana** (sütun bazlı) birleştirir.

A. INNER JOIN (Kesişim)

Her iki tabloda da karşılığı olan kayıtları getirir. Eğer bir müşteri hiç sipariş vermediyse, Inner Join sonucunda listelenmez.

SQL

```
SELECT musteriler.ad, siparisler.tutar
```

```
FROM musteriler
```

```
INNER JOIN siparisler ON musteriler.id = siparisler.musteri_id;
```

B. LEFT JOIN (Sol Dış Birleştirme)

Veri biliminde en çok kullanılan Join türüdür. Soldaki tablonun (ilk tablo) tüm kayıtlarını getirir; sağdaki tabloda karşılığı yoksa NULL değerini atar.

- *Neden Önemli?* Kayıp verileri (analiz dışı kalan müşterileri vb.) görmek için kritiktir.

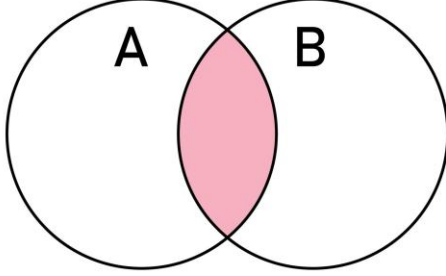
C. RIGHT JOIN

Sağdaki tablonun tüm kayıtlarını getirir. Genellikle Left Join'in tersidir ve çoğu veri bilimci okunabilirlik için tabloların yerini değiştirip Left Join kullanmayı tercih eder.

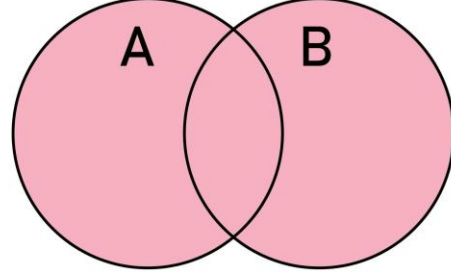
D. FULL OUTER JOIN

Her iki tablodaki tüm kayıtları birleştirir. Eşleşme olmayan yerlere NULL koyar. Veri setindeki tüm "boşlukları" görmek için kullanılır.

Intersection



Union



Shutterstock

3.3. Özel Join Durumları

A. Self Join (Kendine Birleştirme)

Bir tablonun kendisiyle birleştirilmesidir. Genellikle hiyerarşik verilerde (Örn: çalışan-yönetici ilişkisi) kullanılır.

SQL

```
SELECT a.calisan_adi, b.calisan_adi AS yonetici_adi
```

```
FROM calisanlar a
```

```
JOIN calisanlar b ON a.yonetici_id = b.id;
```

B. Cross Join (Kartezyen Çarpım)

Bir tablodaki her satırı, diğer tablodaki tüm satırlarla eşleştirir. Genellikle tüm olasılıkları (Örn: tüm ürünlerin tüm mağazalardaki kombinasyonu) oluşturmak için kullanılır.

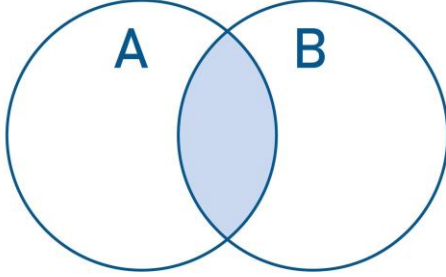
3.4. SET Operatörleri: Dikey Birleştirme (Unions)

Join'ler tabloları yan yana birleştirirken, Union'lar **alt alta** (satır bazlı) birleştirir.

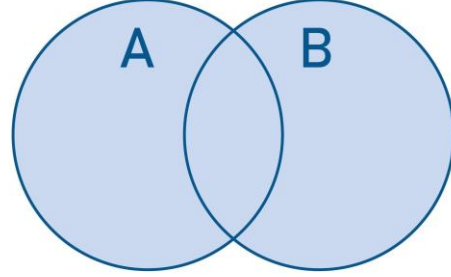
- **UNION:** İki sorgunun sonucunu birleştirir ve **tekrar eden satırları siler** (Unique).
- **UNION ALL:** Tekrar eden satırlara bakmaksızın tüm sonuçları alt alta ekler.

- *Performans Notu:* UNION ALL, UNION'dan daha hızlıdır çünkü SQL'in tekrar edenleri ayıklamak için ekstra bir işlem yapmasına gerek kalmaz.

Intersection



Union



Kural: Birleştirilen sorguların sütun sayıları ve veri tipleri aynı olmalıdır.

3.5. Veri Bilimi İçin Kritik Uyarılar (Pro Tips)

1. Data Duplication (Veri Çoğalması)

Join yaparken dikkat edilmezse, birincil tabloda tek olan bir satır, karşı tabloda birden fazla eşleşme bulursa "çoğalabilir". Bu durum, SUM veya AVG gibi hesaplamaların yanlış çıkmasına neden olur.

İpucu: Join işleminden önce ve sonra satır sayılarını (COUNT(*)) kontrol etmek iyi bir pratiktir.

2. Çoklu Join İşlemleri

Üç veya daha fazla tabloyu birleştirirken SQL motoru bunları sırayla işler. Karmaşıklığı önlemek için Alias (takma isim) kullanımı zorunludur.

```
SELECT m.ad, s.tutar, u.urun_adi
FROM musteriler m
LEFT JOIN siparisler s ON m.id = s.musteri_id
LEFT JOIN urunler u ON s.urun_id = u.id;
```

3. NULL Değerlerin Yönetimi

Join sonrasında oluşan NULL değerleri, modelleme aşamasında hataya yol açabilir. COALESCE fonksiyonu ile bu NULL değerleri anlamlı varsayılan değerlere (Örn: 0) dönüştürebilirsin.